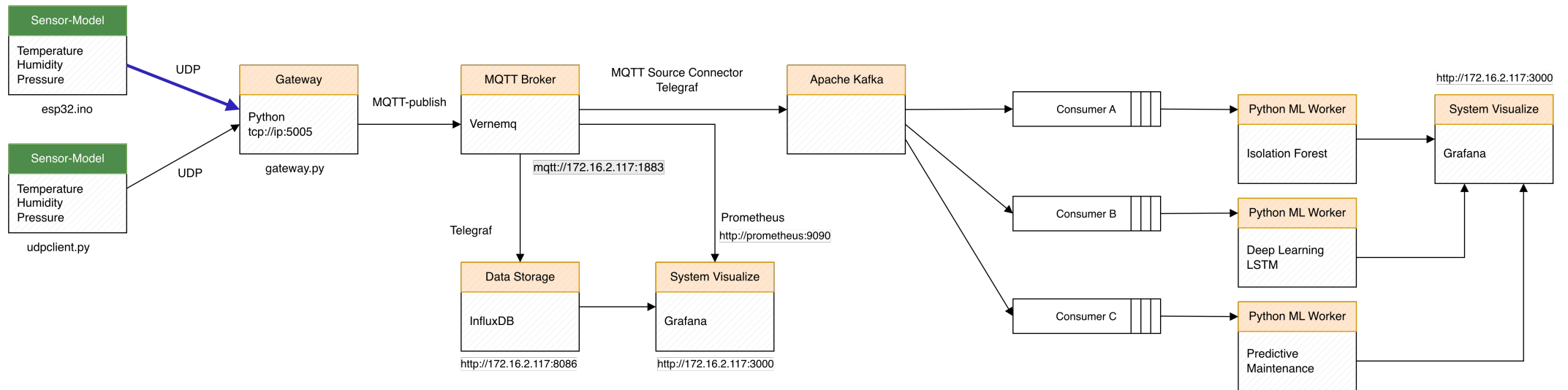


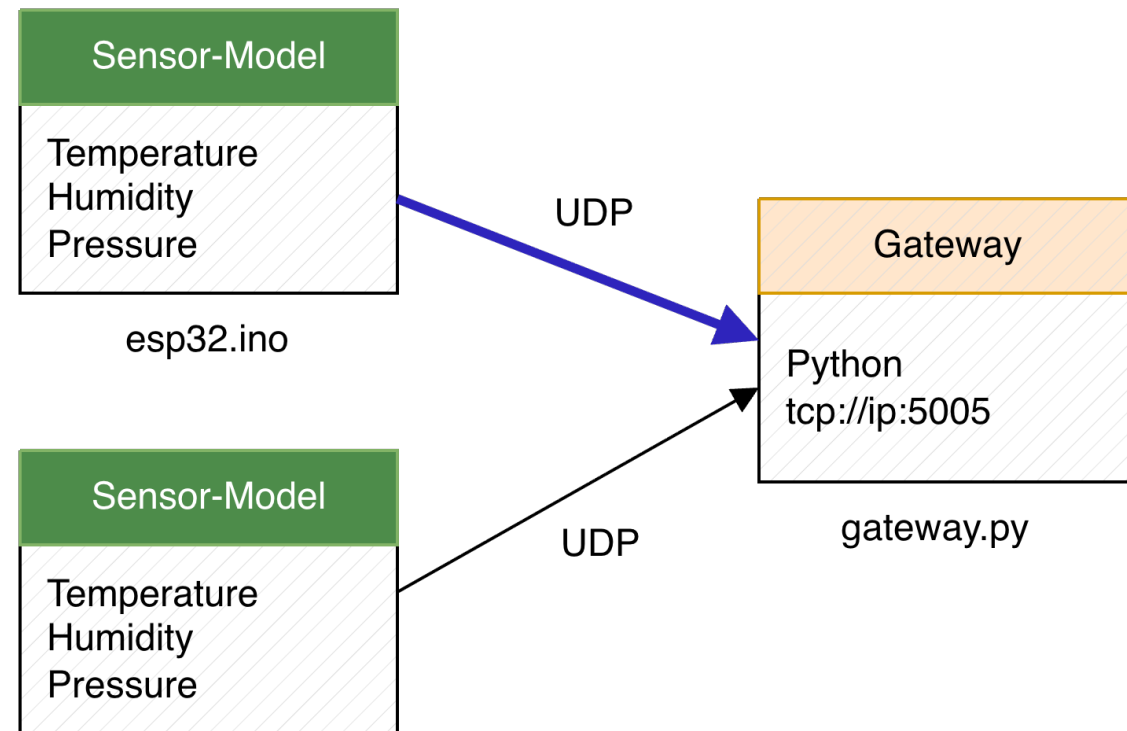
Reverse Engineering IoT

System Architecture



Sensor Level (Edge Nodes)

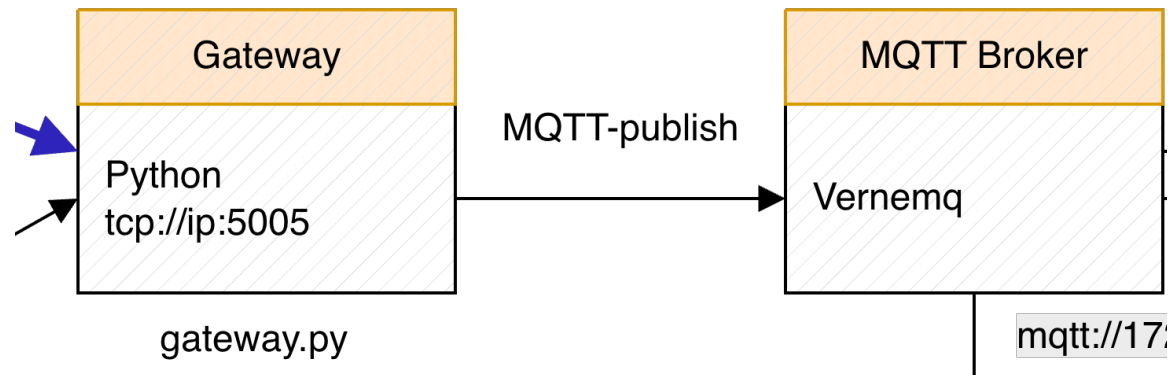
- Device/Script
 - **ESP32 Node** รันด้วยไฟล์ esp32.ino อ่านค่า Temperature, Humidity, Pressure
 - **Python Client Node** รันด้วยไฟล์ udpclient.py จำลอง/ส่งค่า Sensor ชุดเดียวกัน
- การสื่อสาร ส่งข้อมูล sensing data ผ่านโปรโตคอล **UDP** (พอร์ต 5005) ไปยัง Gateway



Gateway Level (Edge Gateway)

- **ประมวลผล**

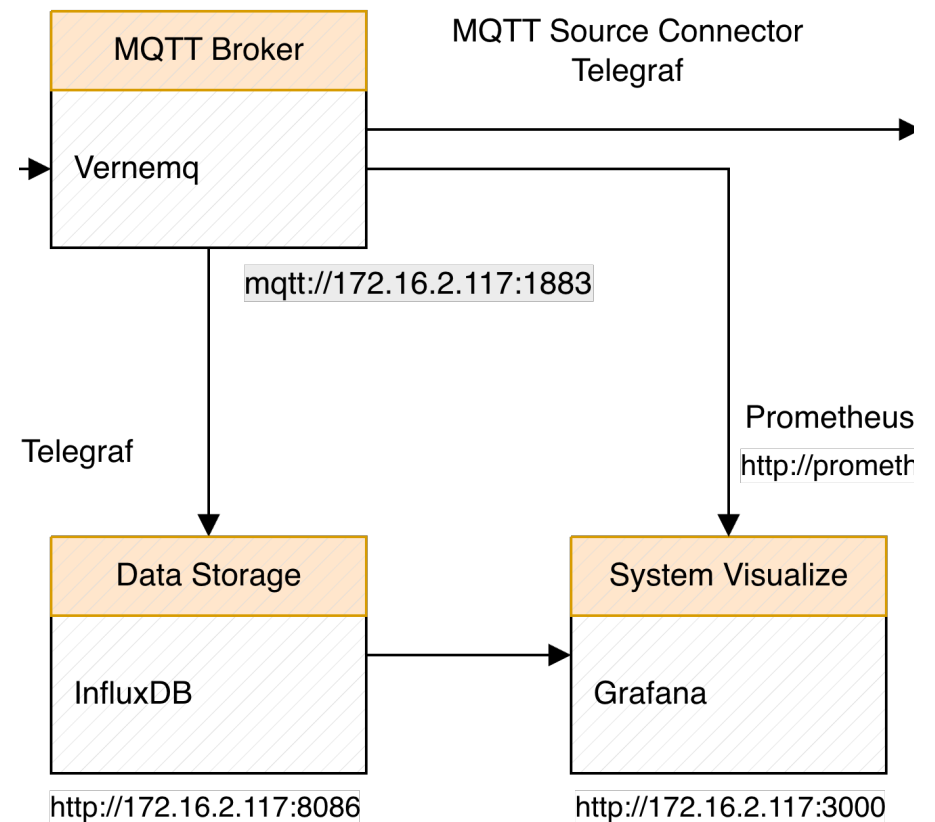
- Gateway.py รับข้อมูล UDP ที่เข้ามา
- Telegraf ที่ทำหน้าที่เป็น Gateway Agent (Collect) ประมวลผล (Process) และส่งต่อ (Forward)



- **การสื่อสาร** แปลงรูปแบบข้อมูลแล้วส่งต่อ (Publish) เข้าสู่ MQTT Broker ผ่านโปรโตคอล MQTT

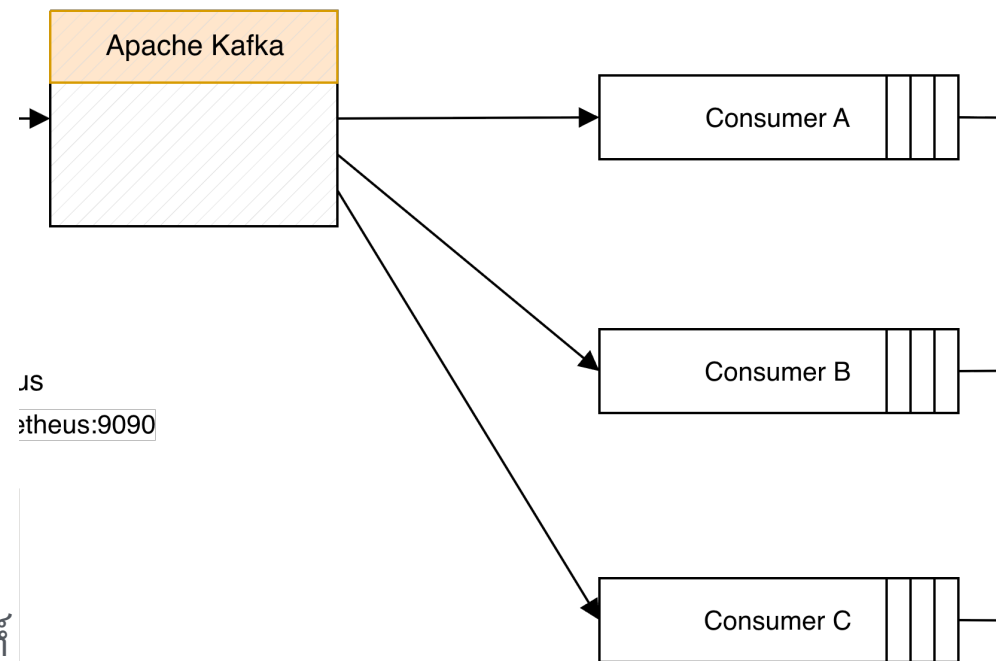
MQTT Broker Level (Messaging/Broker)

- ใช้ **VerneMQ** (พอร์ต 1883) ทำหน้าที่เป็น Broker รับ-ส่งข้อความ central hub
 - เส้นทางแรก (**Raw Data Metrics**) ส่งข้อมูลผ่าน Telegraf เข้า InfluxDB (พอร์ต 8086) เพื่อบันทึกเป็น Time-series Data และแสดงผลข้อมูลบน Grafana (พอร์ต 3000) รวมถึงใช้ Prometheus (พอร์ต 9090) ในการดึง Metric ของ MQTT
 - เส้นทางที่สอง (**Event Streaming Pipeline**) ใช้ Telegraf (MQTT Source Connector) ดึงข้อมูลจาก VerneMQ เพื่อส่งต่อไปยัง Apache Kafka



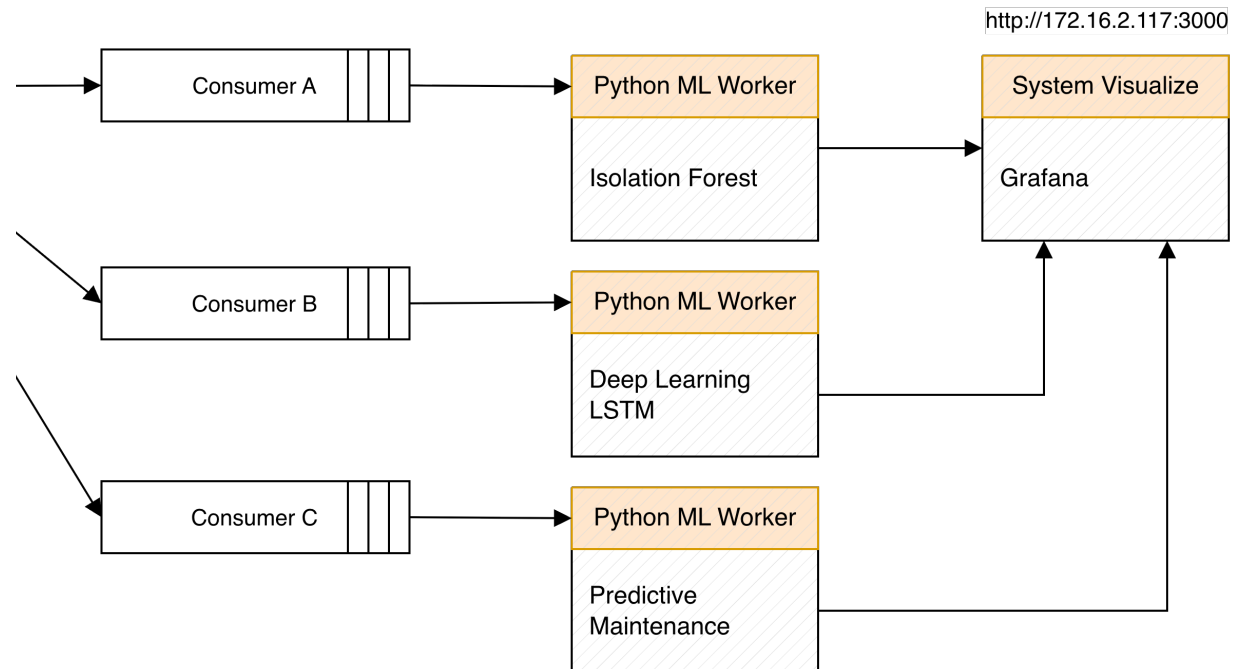
Stream Processing & Machine Learning Layer

- ข้อมูลใน Apache Kafka ถูกกระจายออกเป็น 3 **Consumer Groups/Queues** แบบขนาน เพื่อส่งให้ Python ML Workers ประมวลผล 3 งานหลัก
- Consumer A → **Isolation Forest** ตรวจจับความผิดปกติของข้อมูล (**Anomaly Detection**)
- Consumer B → **Deep Learning LSTM** ทำการพยากรณ์ข้อมูลอนุกรมเวลา (Time-series Forecasting)
- Consumer C → **Predictive Maintenance** วิเคราะห์และคาดการณ์เวลา/โอกาสในการซ่อมบำรุงเครื่องจักร



System Visualization Layer

- ผลลัพธ์จากการประมวลผลของโมเดล ML ทั้ง 3 จะถูกส่งไปแสดงผลแบบเรียลไทม์ร่วมกันบน Grafana (พอร์ต 3000) เพื่อให้ผู้ดูแลระบบติดตามวิเคราะห์ Dashboard ทั้งข้อมูลและผลการทำ ML ได้ในจุดเดียว



Reverse Engineering

การเรียนรู้แบบ Forward (สร้างขึ้น
มา) ทำให้เรา ใช้งานเทคโนโลยีเป็น
แต่การเรียนรู้และนำมาใช้แบบ
Reverse (ย้อนกลับ) จะทำให้เรา วาง
สถาปัตยกรรมระบบ (Architect) เป็น



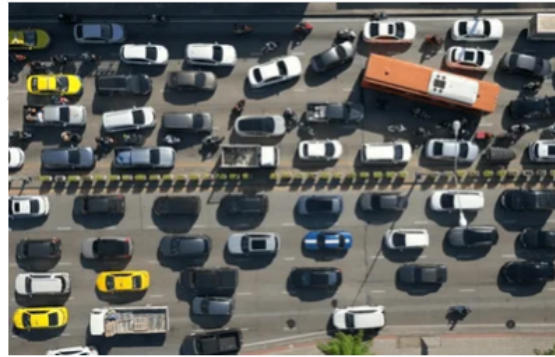
Bird's Eye View (Top-Down/Angled View)



Reverse Engineer in IoT System

1. **Output & Action** ปลายทาง ที่ผู้ใช้งานต้องการ
2. **Analytics & ML Layer** การประมวลผลข้อมูล
3. **Data Pipeline & Message Streaming** ระบบลำเลียงข้อมูล
4. **Edge Computing & Gateways** จุดประมวลผลต้นทาง
5. **Sensing & Data Acquisition** ต้นน้ำ - เก็บข้อมูล

1. Output & Action **ปลายทางที่ผู้ใช้งานต้องการ**



- เป้าหมาย Dashboard

- สรุปความหนาแน่นจราจร

- Alert การกระทำผิดกฎจราจร (จอดในที่ห้ามจอด, วิ่งสวนทาง, สปีดเกินกำหนด) แบบ Real-time

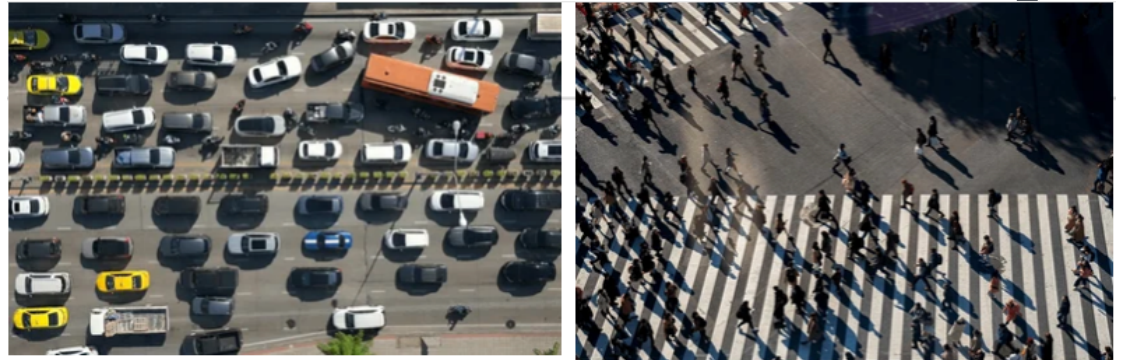
- เครื่องมือ

- Grafana สำหรับ Dashboard แสดงกราฟ/ตัวเลขสถิติ) ร่วมกับ

- Web Application / Notification API (ส่งภาพเหตุการณ์แบบ Real-time เข้ากลุ่มเมื่อพบการกระทำผิด)

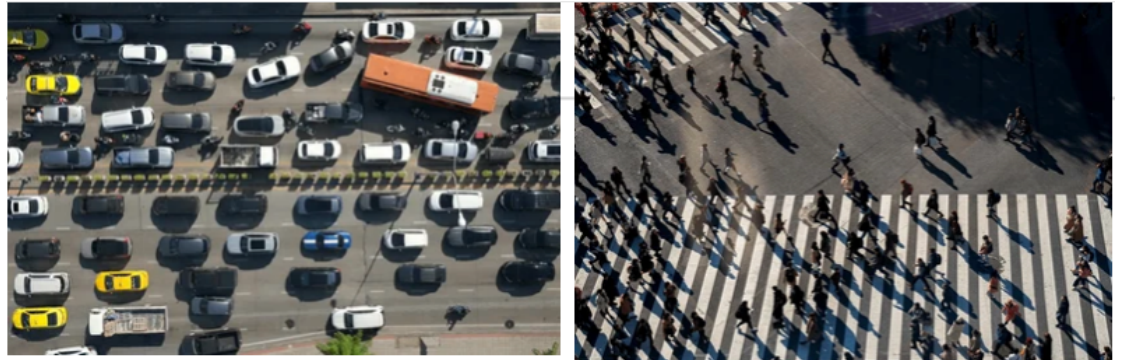
2. Analytics & ML Layer การประมวลผลข้อมูล

โจทย์ที่ต้องใช้ Machine Learning / Computer Vision



- **Object Detection & Classification** ใช้ YOLO (เช่น YOLOv8) เพื่อระบุประเภทรถ (Car, Truck, Bus, Motorbike)
- **Object Tracking** ใช้ **DeepSORT** หรือ **ByteTRACK** เพื่อติดตามรถแต่ละคัน (ให้ ID แก่รถ) ช่วยในการตรวจจับการจอดนิ่ง (Parked) หรือการวิ่งสวนทาง (Wrong-way)
- **Speed Estimation** คำนวณความเร็วจาก ระยะทางจริง ในเฟรม (Pixels to Meters Calibration) ÷ เวลาที่ใช้เคลื่อนที่ระหว่างเฟรม
- **Database** สำหรับเก็บผลลัพธ์ InfluxDB หรือ PostgreSQL เพื่อบันทึก Metadata (ประเภทรถ, ความเร็ว, Timestamp, Bounding Box coordinates)

3. Data Pipeline & Message Streaming ระบบลำเลียงข้อมูล



- โจทย์
 - ภาพวิดีโอถูกประมวลผลได้ผลลัพธ์เป็น **JSON Data** จำนวนมาก ต้องส่งข้อมูลเข้า Database และ Alert แบบไม่ให้ระบบล่ม
- เทคโนโลยี
 - Apache Kafka รับ Event Data จากโมเดล ML กระจายไปยัง **Consumer** สองทาง (ทางหนึ่งลง InfluxDB อีกทางส่ง Alert)

4. Edge Computing & Gateways จุดประมวลผลต้นทาง



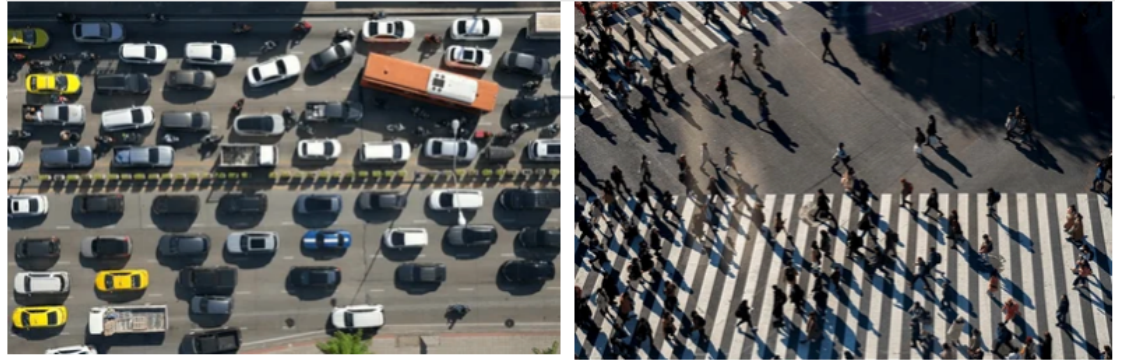
- โจทย์

- วิดีโอสตรีมมีความจุสูงมาก (High Bandwidth) ไม่ควรถ่ายโอนไฟล์วิดีโอทั้งหมดขึ้น Cloud

- เทคโนโลยี/อุปกรณ์

- **Edge AI Node** บอร์ดอย่าง NVIDIA Jetson Nano / Orin หรือ PC ที่จุดตรวจ เพื่อรันโมเดล **YOLO + Tracking** ที่ฝั่ง Edge
- **Gateway** ส่งเฉพาะ **Metadata/JSON** (เช่น {"vehicle_id": 102, "type": "car", "speed": 85, "status": "speeding"}) ผ่านโปรโตคอล **MQTT** หรือ **UDP** เข้าสู่ Pipeline หลัก

5. Sensing & Data Acquisition ต้นน้ำ - ฮาร์ดแวร์เก็บข้อมูล



- อุปกรณ์
 - **CCTV / IP Camera** (RTSP Stream) สำหรับดึงภาพการจราจร คน วัตถุ
 - (Optional) **Radar Sensor / Ultrasonic** ส่งข้อมูลความเร็วผ่าน **ESP32** ด้วย **UDP** เพื่อเอามา Cross-validate (ยืนยันความแม่นยำ) ร่วมกับค่าความเร็วที่คำนวณจากภาพวิดีโอ

โจทย์เพื่อออกแบบระบบย้อนกลับ (Reverse Engineering)



- พฤติกรรมการขับขี่ผิดกฎจราจร (Traffic Violations)
- สภาพการจราจรและผังเมือง (Traffic Analytics & Smart City)
- ความปลอดภัยและเหตุการณ์ฉุกเฉิน (Safety & Incident Detection)

พฤติกรรมการขับขี่ผิดกฎจราจร (Traffic Violations)



- **ฝ่าสัญญาณไฟแดง (Red Light Running)**

- ML/Logic กำหนด Zone เส้นหยุด (Stop Line) + ตรวจจับสถานะไฟจราจร (หรือดึง Trigger จากตู้ไฟ) หาก Bounding Box ของรถข้ามเส้นขณะไฟแดง ให้ทำ Event Alert

- **เปลี่ยนเลนในเขตเส้นทึบ (Illegal Lane Change / Solid Line Crossing)**

- ML/Logic ใช้ Hough Line Transform หรือ Segmentation วาดเส้นจราจรบนถนน หาก Vector เคลื่อนที่ของรถตัดผ่านเส้นทึบจะนับเป็น Violation

- **ไม่จอดให้คนข้ามทางมาลาย (Pedestrian Right-of-Way Violation)**

- ML/Logic ตรวจจับทั้ง Vehicle และ Pedestrian หากคนกำลังเดินอยู่ใน Crosswalk Zone แล้วรถไม่ชะลอ/ขับผ่านในระยะกระชั้นชิด ให้ส่ง Notification

การวิเคราะห์สภาพการจราจรและผังเมือง (Traffic Analytics & Smart City)



- **การนับจำนวนรถแยกประเภท (Vehicle Counting & Classification)**
 - ML/Logic นับจำนวนรถที่ผ่านแยกแบ่งตามประเภท (Car, Motorcycle, Bus, Truck) แยกตามช่วงเวลา เพื่อนำไปวิเคราะห์ปริมาณการใช้ถนน
- **ตรวจจับความหนาแน่นและสภาวะรถติด (Traffic Congestion Index / Occupancy)**
 - ML/Logic คำนวณ % พื้นที่ของถนนที่ถูกปกคลุมด้วย Bounding Box ของรถ หากเกิน Threshold ที่กำหนด (เช่น >80% เป็นเวลาเกิน 3 นาที) ให้ปรับสถานะไฟจราจรหรือแจ้งเตือนจราจรติดขัด
- **การวัดระยะเวลาคิวรถแยก (Queue Length & Waiting Time Estimation)**
 - ML/Logic นับจำนวนรถที่หยุดนิ่งในแต่ละเลนบริเวณติดสัญญาณไฟ เพื่อคำนวณเวลาที่รถต้องจอดรอเฉลี่ย นำไปใช้อัลกอริทึมปรับเวลาไฟเขียว-ไฟแดงแบบไดนามิก (Adaptive Traffic Signal Control)

ความปลอดภัยและเหตุการณ์ฉุกเฉิน (Safety & Incident Detection)



- **ตรวจจับอุบัติเหตุเฉี่ยวชน (Accident / Collision Detection)**
 - ML/Logic ตรวจจับเมื่อ Bounding Box สองอัน Overlap กันอย่างรวดเร็ว + Vector ความเร็วเปลี่ยนเป็น 0 ทันทีทันใด (Sudden Stop) พร้อมส่งภาพ Snapshot ไปยังศูนย์ควบคุม
- **การเปิดทางให้รถฉุกเฉิน (Emergency Vehicle Priority)**
 - ML/Logic เทรน Class เพิ่มเติมสำหรับ Ambulance / Firetruck เมื่อตรวจพบรถฉุกเฉินกำลังมุ่งหน้ามายังแยก ให้ส่ง Signal ไปควบคุมระบบไฟจราจรเปิดไฟเขียวให้อัตโนมัติ

Reverse Engineer Example

การตรวจจับรถฝ่าสัญญาณไฟแดง (Red Light Violation)

- **Output (Dashboard & Alert)**

- ภาพ Snapshot ของรถพร้อมป้ายทะเบียน + คลิปสั้น 3 วินาทีช่วงเกิดเหตุ + ค่า timestamp / ทะเบียน / ความเร็ว
- ส่งเข้า Grafana Dashboard สำหรับดูสถิติ และส่ง Line / Telegram Notification ถึงเจ้าหน้าที่

Reverse Engineer Example

การตรวจจับรถฝ่าสัญญาณไฟแดง (Red Light Violation)

- **ML & Logic Layer**

- **YOLOv8** Detect ประเภทรถและ ROI ของไฟจราจร
- **ByteTRACK** ติดตาม ID รถเพื่อดูทิศทางการเคลื่อนที่ตัดผ่าน Line Trigger
- **Database** บันทึกรูปภาพลง disk และเก็บ Metadata ลง InfluxDB/PostgreSQL

Reverse Engineer Example

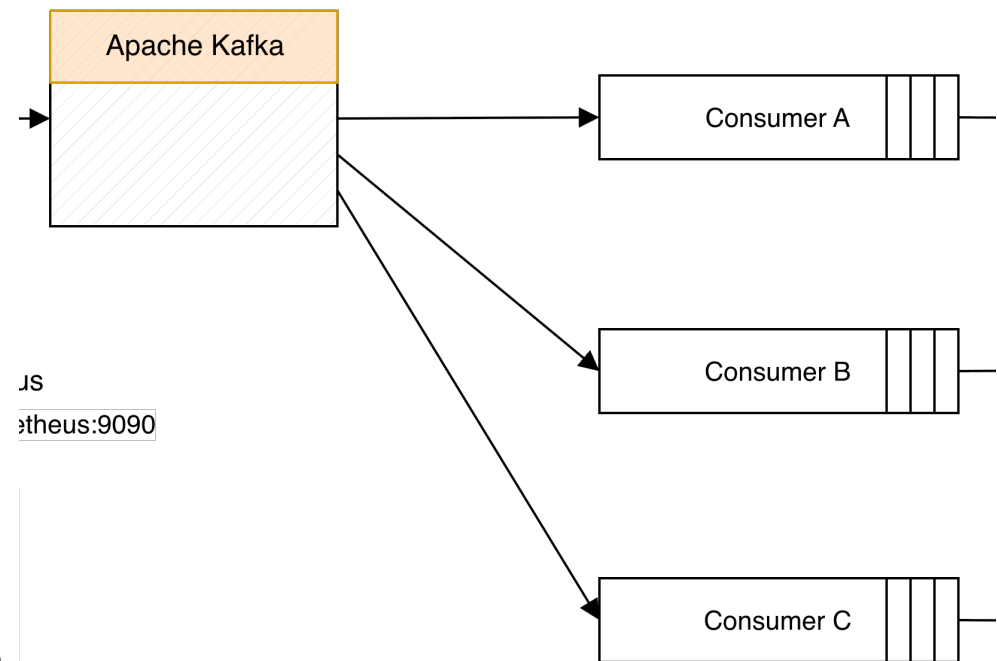
การตรวจจับรถฝ่าสัญญาณไฟแดง (Red Light Violation)

- **Pipeline Layer**

- Apache Kafka รับ Event เมื่อเกิดเหตุฝ่าไฟแดง เพื่อแยก Consumer ฝั่งลง DB และฝั่ง Notification ไม่ให้บล็อกการทำงานกันเอง

- **Edge & Sensing Layer**

- Camera (RTSP Stream) ส่งภาพ **1080p@30fps** เข้า Edge Server (NVIDIA Jetson / GPU Node) บริเวณใกล้สี่แยก เพื่อประมวลผล real-time ไม่ต้องส่งวิดีโอขึ้น Cloud



Mini Project

Edge-to-Cloud Real-Time Smart Traffic Analytics Platform

ระบบวิเคราะห์การจราจรอัจฉริยะและตรวจจับการกระทำผิดกฎจราจรแบบเรียลไทม์

- ให้นักเรียนออกแบบและพัฒนาโครงสร้างพื้นฐาน IoT แล้ววางระบบลำเลียงข้อมูล (Data Pipeline) จากภาพวิดีโอจราจร ไปจนถึงการประมวลผล ML และแสดงผลลัพธ์
- ข้อกำหนดการทำงานจำแนกตาม
 - Output & Presentation Layer
 - ML & Logic Layer
 - Pipeline Layer
 - Edge & Sensing Layer
 - Documentation & Delivery

Edge-to-Cloud Real-Time Smart Traffic Analytics Platform

Output & Presentation Layer

- สร้าง **Grafana Dashboard** ด้วยตัวเลือก ดังนี้
 - การนับจำนวนรถแยกประเภท (Vehicle Counting & Classification)
 - ตรวจจับความหนาแน่นและสภาวะรถติด (Traffic Congestion Index / Occupancy)
 - การวัดระยะเวลาคิวสี่แยก (Queue Length & Waiting Time Estimation)
 - พฤติกรรมการขับขี่ผิดกฎจราจร (Traffic Violations)
- ระบบ **Alert** หรือบันทึกเหตุการณ์ ทันทีเมื่อเกิดเหตุการณ์กระทำผิดกฎจราจร

Edge-to-Cloud Real-Time Smart Traffic Analytics Platform

ML & Logic Layer

- ใช้ **YOLO** ทำ Object Detection & Classification เพื่อระบุประเภทยานพาหนะ
- ใช้ **Object Tracking** (เช่น DeepSORT/ByteTRACK) เพื่อติดตามตำแหน่งรถ และคำนวณหา 3 เหตุการณ์หลัก:
- **Speeding Detection**: คำนวณความเร็วรถเทียบกับระยะทางจริง หากเกิน Threshold ให้ trigger Alert
- **Wrong-way Driving**: ตรวจสอบทิศทางการเคลื่อนที่ (Vector) ของ Bounding Box หากสวนทางกับเลนถนนที่กำหนด ให้ trigger Alert
- **Illegal Parking**: ตรวจสอบรถที่หยุดนิ่งในโซนห้ามจอด (No-Parking Zone) เกินเวลาที่กำหนด ให้ trigger Alert

Edge-to-Cloud Real-Time Smart Traffic Analytics Platform

Pipeline Layer

- ใช้ Apache **Kafka** ทำหน้าที่เป็น Event Streaming Broker รองรับผลลัพธ์ JSON ที่ส่งมาจาก ML Engine โดยแบ่ง Consumer ออกเป็นอย่างน้อย 2 สาย (สายบันทึกลง Database และสายยิง Alert Notification)
- จัดเก็บข้อมูลแบบ Time-series ลงใน InfluxDB หรือ PostgreSQL เพื่อนำไปต่อเข้ากับ Grafana

Edge-to-Cloud Real-Time Smart Traffic Analytics Platform

Edge & Sensing Layer

- รับข้อมูลวิดีโอ (RTSP Stream หรือไฟล์วิดีโอจำลองภาพมุมสูงจากสี่แยก/ตึกชั้น 2)
- ประมวลผลวิดีโอฝั่ง Edge (Python Script) แปลงผลลัพธ์การตรวจจับเป็น JSON Payload แล้วส่งผ่านโปรโตคอล MQTT หรือ UDP เข้าสู่ Gateway/Pipeline หลัก

Edge-to-Cloud Real-Time Smart Traffic Analytics Platform

Documentation & Delivery

- จัดทำ System Architecture Diagram แสดงการไหลของข้อมูล
- อัปโหลด Source Code ทั้งหมดขึ้น GitHub พร้อมมี README.md อธิบายขั้นตอนการตั้งค่า และการรันระบบด้วย Docker Compose (ถ้ามี)

ตัวอย่าง JSON Payload Structure ที่ต้องตกลงกันก่อนเขียนโค้ด (Data Contract)

JSON

```
{  
  "timestamp": "2026-08-18T16:00:00Z",  
  "camera_id": "CAM_BUILDING2_FL02",  
  "vehicle_id": 1042,  
  "type": "car",  
  "speed_kmh": 85.5,  
  "event_type": "SPEEDING_VIOLATION",  
  "location": {"zone": "Lane_A", "x": 120, "y": 450},  
  "snapshot_url": "http://minio-server/snapshots/viol_1042.jpg"  
}
```

เกณฑ์การให้คะแนน 500 คะแนน

• Output & Presentation	• Dashboard Grafana	• 100
• ML & Logic Layer	• Model Performance, Tracking & Event Rules, Accuracy	• 125
• Pipeline Layer	• Streaming (Kafka), Data Storage (InfluxDB), Broker	• 100
• Edge & Sensing Layer	• Camera Input, Edge Processing, Protocol (MQTT/UDP)	• 75
• Documentation & Code	• Architecture Diagram, Code Quality, User Manual	• 100